

G.A.T.E.S. — Gate 1

What it checks, what it measured, and what changed when it was switched on

Generated 2026-08-15 · gates @ main · 249 gate tests, 63 host integration tests · every number below is a recorded measurement or a cited published figure

1. The defect, in one line of the host scaffold

```
def execute_code(code_str, timeout=60, MAX_LEN=1000):
    ...
    except Exception as e:
        output_capture.write(f"[CODE EXECUTION ERROR]: {str(e)}")
    # appended AFTER the prints
    return output_capture.getvalue()[:MAX_LEN] # ...then sliced off
```

The writing agent's entire experimental record was 1,000 characters, and the crash marker was appended after the program's own output — so on any run that printed more than that, the marker fell off the same slice and the solver's only crash test never fired.

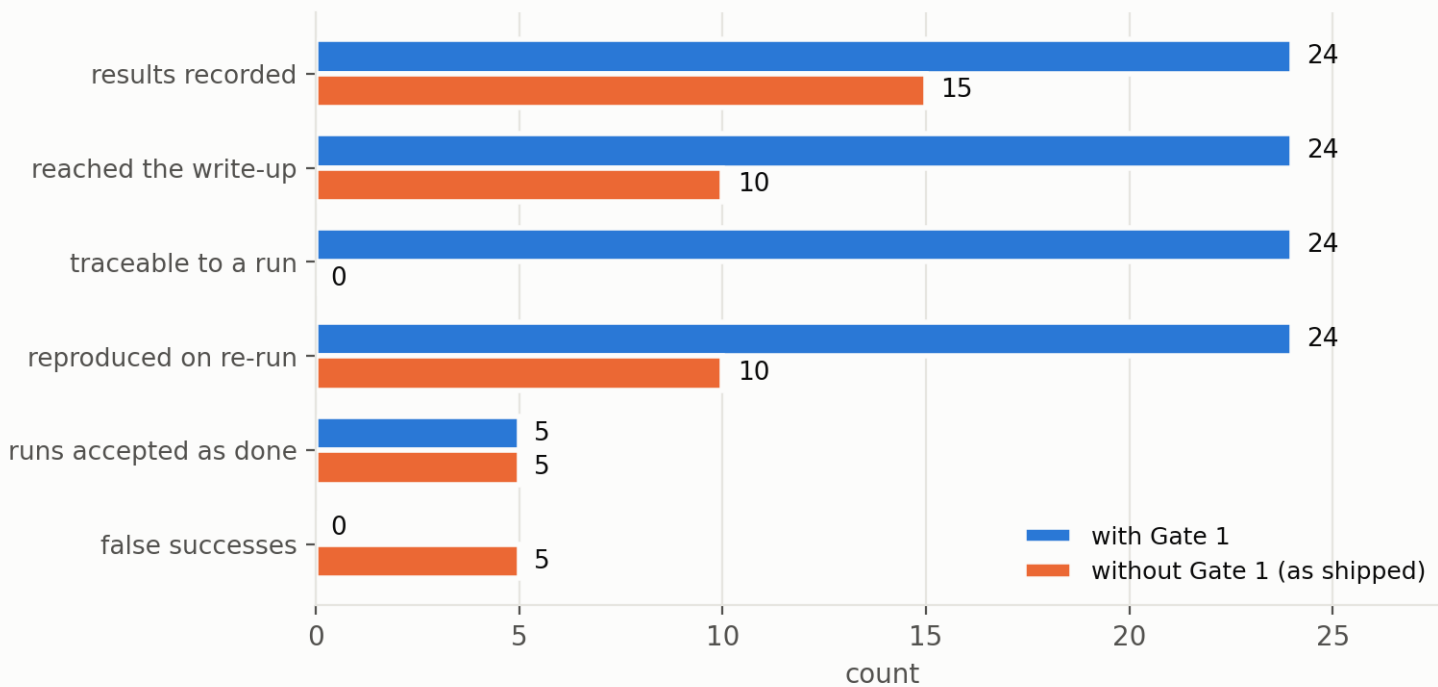
measured in the archived run	value
Reward score on a run raising <code>NameError</code> every attempt	0.95 → 0.98 → 1.0
Test accuracy reported in the paper	81.60% (never measured)
Speedup reported	13.61× (never measured)
Ablation collapse reported	39.20% (never measured)
Archived runs usable of seven	1

2. The comparison: Agent Laboratory with and without Gate 1

This is the whole argument in one figure. Both arms run the same task, the same engineer model, the same temperature and the same turn budget; they differ only in what is allowed to decide that an experiment succeeded.

Agent Laboratory, with and without Gate 1

5 runs, data-efficiency task, deepseek-v4-flash engineer



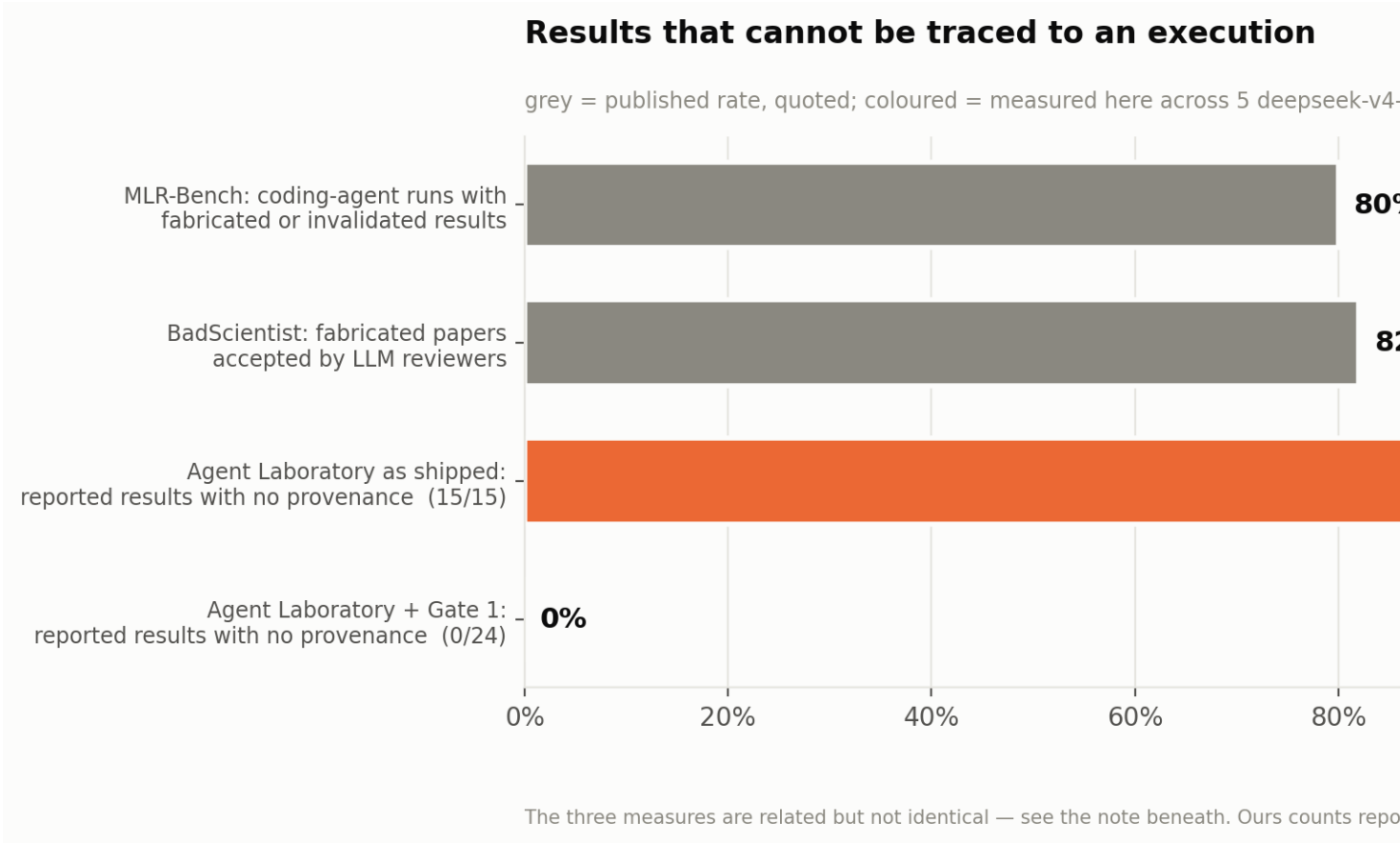
Both arms accept every run. Acceptance is not where they differ — everything that happens to the numbers

afterwards is. With Gate 1, **100%** of what was reported reproduced on re-execution and all of it carries a trace

id. Without, **0** values were recorded through any contract, five of fifteen results never reached the write-up at all, and none of them can be tied to the run that produced them.

2.1 Against the rates the benchmark papers report

"Research agents fabricate results" is not a claim this report needs to establish — the field has quantified it. **MLR-Bench** (arXiv 2505.19955) finds that current coding agents produce fabricated or invalidated experimental results in **80%** of cases, and that both agents it evaluates score below its 6.0 acceptance threshold on *Soundness* while scoring respectably on *Completeness* — fluent, and unsound. **BadScientist** (arXiv 2510.18003) finds deliberately fabricated papers accepted at rates up to **82%**, with detection "barely exceeding random chance", and names the defence it thinks is needed: *provenance verification*. That is what Gate 1 is.



The three bars are related but not the same construct, and the chart says so rather than implying a like-for-like ranking. MLR-Bench's is a share of runs; BadScientist's is an acceptance rate under an LLM reviewer; ours is the share of reported results that no execution can be shown to have produced. What they have in common is the failure they measure.

2.2 The task, and one run in detail

Research question given to the agent: how much labelled data does a classifier need? Make a synthetic 3-class dataset, train multinomial logistic regression by gradient descent on 25% of the training samples and on all of them, and report the two test accuracies and their ratio.

Models. Read from the 5 run record(s) behind every number in this report, not asserted. The ML engineer — the agent whose work is being judged — runs on deepseek-v4-flash. Gate 1's two model roles run on qwen3-8b-local, locally and at no per-call cost; they read logs and write the feedback report, and are never consulted for the verdict. Both arms use the same engineer model at the same temperature, so neither is advantaged.

arm	accepted a run	turns used	numbers reported	numbers checkable	reproduced	reproduction rate
with Gate 1	yes	2	6	6	6	100%
without Gate 1	yes	1	0	0	0	—

The accepted run recorded no values, so none of the numbers it would hand the writer can be checked against an execution.

2.3 The numbers themselves

metric	with Gate 1	re-executed	verdict	without Gate 1
--------	-------------	-------------	---------	----------------

	reported	same code, fresh process	reported
eff.acc_at_100	1	1	reproduced not recorded
eff.acc_at_25	1	1	reproduced not recorded
eff. efficiency_ratio	1	1	reproduced not recorded
logreg_25_test_acc	1	1	reproduced not recorded
logreg_full_test_acc	1	1	reproduced not recorded
logreg_full_vs_25_acc_ratio	1	1	reproduced not recorded

2.4 Turn by turn

arm	turn	verdict	other arm	exit	stdout bytes	values recorded	failing checks
with Gate 1	1	rejected	would accept	0	1,397	3	results.expected_keys_present
with Gate 1	2	accepted	would accept	0	1,485	6	—
without Gate 1	1	accepted	would reject	0	693	0	results.contract_present

3. Across every run on disk

One run is an anecdote. Every ablation record found under `/home/kesh/AgentLaboratory-Gemini/ablation_runs` is aggregated here; a run that predates a measurement is reported as *not measured* rather than as zero.

across 5 run(s)	with Gate 1	without Gate 1
accepted a run	5 of 5	5 of 5
mean turns used	2.0	1.0
numbers reported in total	24	0
numbers checkable against an execution	24	0
reproduced on re-execution	24 (100%)	0 (—)
false successes — accepted what the other arm rejected	0	5

run	engineer	temp	with Gate 1				without Gate 1			
			turns	accepted	reported	reproduced	turns	accepted	reported	false success
data_efficiency	deepseek-v4-flash T=?	2	yes	6	6	6	1	yes	0	1
de_1	deepseek-v4-flash T=0.7	2	yes	3	3	3	1	yes	0	1
de_2	deepseek-v4-flash T=0.7	2	yes	6	6	6	1	yes	0	1

de_3	deepseek-v4-flash T=0.7	2	yes	6	6	1	yes	0	1
de_4	deepseek-v4-flash T=0.7	2	yes	3	3	1	yes	0	1

3.1 What each arm's report actually says

Both arms produce a report; they are measured here the same way, on three properties that are genuinely different and are kept apart because collapsing them would hide the real result. **Completeness** — did the result reach the writing agent at all? **Traceability** — does anything bind it to the execution that produced it? **Accuracy** — does it reproduce when the accepted code is re-run? For the gated arm the report is its registry. For the ungated arm it is whatever numbers appear in the 1,000 characters upstream hands the writer, because those are the only ones the writer can copy.

run	with Gate 1				without Gate 1				
	produced	reached writer	traceable	reproduced	produced	reached writer	lost to channel	traceable	reproduced
data_efficiency	6	6	6	6	3	3	0	0	3/3
de_1	3	3	3	3	3	3	0	0	3/3
de_2	6	6	6	6	3	1	2	0	1/1
de_3	6	6	6	6	3	3	0	0	3/3
de_4	3	3	3	3	3	0	3	0	0/0
total	24	24	24	24	15	10	5	0	10/10

The ungated arm's numbers are not wrong. Of the 10 that reached the writer and could be checked, 10 reproduced exactly when the same code was re-run. Accuracy is not where that arm fails.

It fails on completeness and traceability. 5 of 15 results the runs actually produced never reached the writing agent at all — they fell outside the 1,000-character window, and in one run all three did, leaving a writer with a training log and no results. And 0 of 15 carry anything binding them to the run that produced them: no trace id, no code hash. A number that reproduces by luck and a number that was measured are indistinguishable to the reader.

With Gate 1, 24 of 24 results reached the writer and 24 of 24 carry a trace id and a code hash. The registry is not truncated, which is the whole point of replacing the channel rather than widening it.

3.2 The papers

Everything above measures a link in the chain. A reader sees none of it — they see a paper. So the last question is the only one that reaches them: of the numbers a generated paper states as its findings, how many can be tied

back to an execution? Both papers are put through the same audit, with the same claim filter and the same rounding tolerance; the only asymmetry is the one being measured, which is that one run has a registry to check against and the other recorded nothing.

paper	numeric claims	sourced to a recorded result	sourced	record_result calls
without Gate 1	65	0	0%	0
with Gate 1	paper not found			

The archived paper states 65 numeric findings and sources none of them. Its saved experiment code calls record_result zero times, and the run that produced it raised NameError on every attempt while scoring 1.0. The paper is fluent, specific, and unfalsifiable from its own artifacts — which is the failure mode a validity layer exists to make impossible rather than unlikely.

A sample of the unsourced claims, quoted as the paper states them:

value	as the paper states it
81.6	..., where peak classification accuracy is achieved at K=2 (81.60 % test accuracy and 80.40 %...
80.4	...accuracy is achieved at K=2 (81.60 % test accuracy and 80.40 % validation accuracy) but d...
59.1	...d 80.40 % validation accuracy) but decays drastically to 59.10 % test accuracy at K=10 as...
81.1	...sification accuracy (matching and slightly exceeding GCN's 81.10 % test accuracy) while deliv...
13.61	...xceeding GCN's 81.10 % test accuracy) while delivering a 13.61 execution speedup (0....
0.018	...uracy) while delivering a 13.61 execution speedup (0.0180 seconds vs. 0.2450 seconds...
0.245	...ng a 13.61 execution speedup (0.0180 seconds vs. 0.2450 seconds) due to its decouple...
8.8	...trick, demonstrating that omitting self-loops leads to an 8.80 % test accuracy drop at K=2...
72.8	...lf-loops leads to an 8.80 % test accuracy drop at K=2 (72.80 %) and a catastrophic collap...
39.2	...y drop at K=2 (72.80 %) and a catastrophic collapse to 39.20 % test accuracy at K=8 . We...
and 55 more.	

The Gate 1 paper is not in this build. paper not found. It is left absent rather than estimated: an empty column here would read as a measurement, and this report has no business making that mistake in a section about unsourced numbers.

4. Every check that ran

Not the catalogue of what Gate 1 *could* check — the record of what it did check, on this run, turn by turn.

check	severity	turn 1	turn 2
static.syntax_valid	FAIL	pass	pass

static.no_unbound_names	FAIL	pass	pass
static.no_banned_calls	FAIL	pass	pass
results.contract_not_shadowed	FAIL	pass	pass
exec.completed_within_budget	FAIL	pass	pass
exec.exit_code_zero	FAIL	pass	pass
exec.no_uncaught_exception	FAIL	pass	pass
env.code_identity	FAIL	pass	pass
exec.no_swallowed_traceback	WARN	pass	pass
logs.no_error_signals	WARN	pass	pass
env.clean_namespace	FAIL	pass	pass
env.seed_recorded	WARN	pass	pass
results.contract_present	FAIL	pass	pass
results.expected_keys_present	FAIL	FAIL	pass
results.values_computed	FAIL	pass	pass
results.values_finite	FAIL	pass	pass
results.single_observation	WARN	pass	pass
results.non_degenerate	WARN	pass	WARN
output.untruncated	INFO	pass	pass
env.provenance	INFO	pass	pass
logs.model_error_signals	WARN	not run	WARN

4.1 What the same checks said about the ungated arm

The ungated arm is judged by upstream's rule, but every one of its executions was also passed through Gate 1 so the two verdicts can be compared on identical evidence. Those shadow verdicts:

check	severity	turn 1
static.syntax_valid	FAIL	pass
static.no_unbound_names	FAIL	pass
static.no_banned_calls	FAIL	pass
results.contract_not_shadowed	FAIL	pass

d

exec.completed_within_budget	FAIL	pass
exec.exit_code_zero	FAIL	pass
exec.no_uncaught_exception	FAIL	pass
env.code_identity	FAIL	pass
exec.no_swallowed_traceback	WARN	pass
logs.no_error_signals	WARN	pass
logs.model_error_signals	INFO	pass
env.clean_namespace	FAIL	pass
env.seed_recorded	WARN	WARN
results.contract_present	FAIL	FAIL
output.untruncated	INFO	pass
env.provenance	INFO	pass

5. What the run cost

role	calls	prompt tokens	completion tokens	of which reasoning	billed
engineer	3	1,755	8,452	5,568	paid API
gate	4	2,426	2,836	0	local — no marginal cost
total	7	4,181	11,288	5,568	1,613 completion tokens per call

Gate 1's marginal cost on the paid API is zero. Its two jobs ran locally: 4 calls, 2,836 completion tokens, nothing billed.

For scale, one engineer call costs 2,817 completion tokens against 709 for a gate call — **4×**. Almost all of the engineer's is reasoning, which is billed and never appears in the output.

What Gate 1 *does* change on the bill is rewrites: each rejection buys another engineer call. In this run **0** attempt(s) were rejected by the static tier before execution, costing no compute at all, and every rejection that names the real fault is a rewrite the agent does not waste — which is the whole reason the shadowed-contract check was added.

6. The feedback the engineer actually received

Turn 1, rejected on `results.expected_keys_present`.

The deterministic finding above. Below, what the LLM layer wrote for the engineer from it — every name in it checked against the report before it was shown:

- 1. In line 45, add `eff.acc\_at\_100` to the metrics dictionary.
- 2. In line 46, add `eff.acc\_at\_25` to the metrics dictionary.
- 3. In line 47, add `eff. efficiency\_ratio` to the metrics dictionary.
- 4. In line 48, add `logreg\_25\_test\_acc`, `logreg\_full\_test\_acc`, and `logreg\_full\_vs\_25\_acc\_ratio` to the metrics dictionary.

7. The logs

arm	path	files per attempt
with Gate 1	/home/kesh/AgentLaboratory-Gemini/ablation_runs/ data_efficiency/gated/gate1/attempt_NN/	experiment.py · stdout.txt · stderr.txt · results.json · registry.json · gate1_report.json
without Gate 1	/home/kesh/AgentLaboratory-Gemini/ablation_runs/ data_efficiency/ungated/attempt_NN/	experiment.py · stdout.txt · stderr.txt
shadow verdicts	/home/kesh/AgentLaboratory-Gemini/ablation_runs/ data_efficiency/ungated/shadow_gate/gate1/attempt_NN/	what Gate 1 would have said about the ungated arm's executions
re-execution	/home/kesh/AgentLaboratory-Gemini/ablation_runs/ data_efficiency/<arm>/verify/	the accepted code, run again
summary	/home/kesh/AgentLaboratory-Gemini/ablation_runs/ data_efficiency/ablation.txt · ablation.json	the tables in this report

7.1 With Gate 1 — captured in full, and the values recorded separately

Step 441, loss: 0.039067
Step 461, loss: 0.037578
Step 481, loss: 0.036209
Step 500, loss: 0.035006
Test accuracy (full training set): 1.0000
Ratio (full/sub): 1.0000

/home/kesh/AgentLaboratory-Gemini/ablation_runs/data_efficiency/gated/gate1/attempt_02/stdout.txt — 57 lines total

7.2 Without Gate 1 — the same capture, but only the first 1,000 characters ever reach the agent

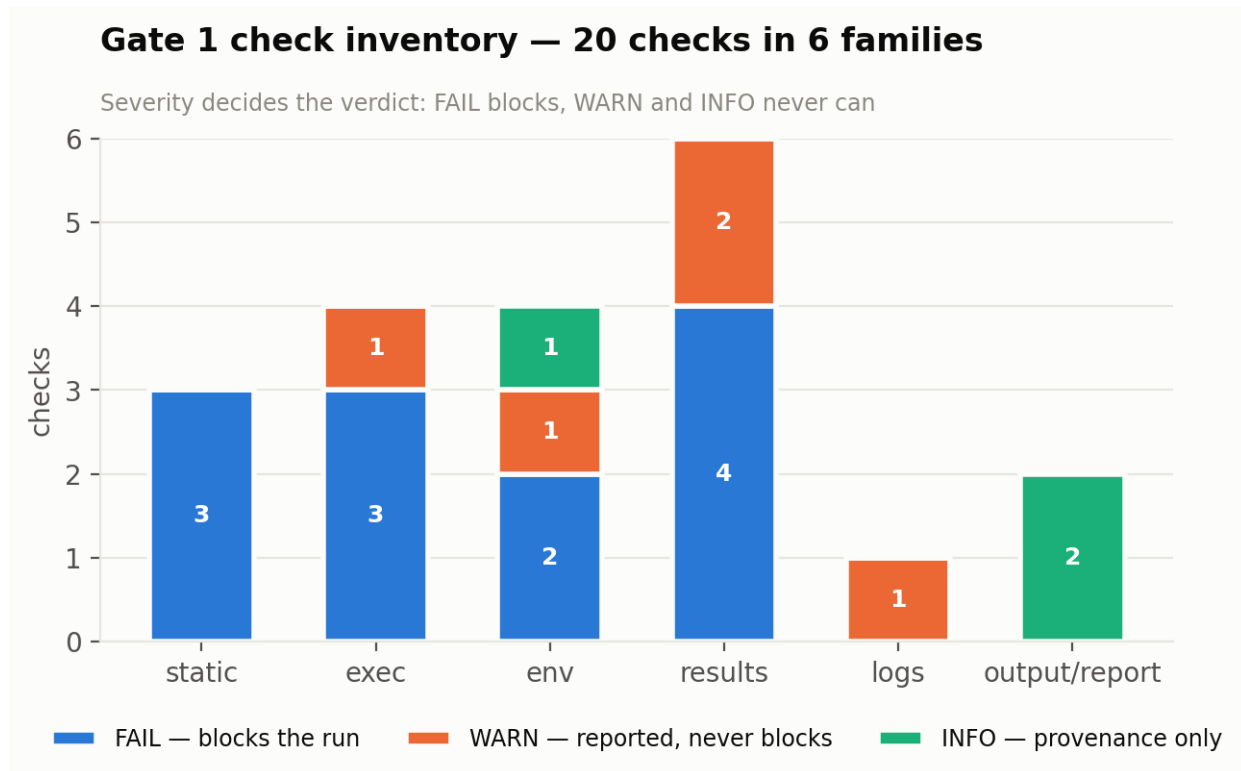
Training on 25% of training samples:
Step 20: loss = 0.056490
Step 40: loss = 0.032105
Step 60: loss = 0.023019
Step 80: loss = 0.018161
Step 100: loss = 0.015100
Step 120: loss = 0.012980
Step 140: loss = 0.011417
Step 160: loss = 0.010213

Step 180: loss = 0.009254

/home/kesh/AgentLaboratory-Gemini/ablation_runs/data_efficiency/ungated/attempt_01/stdout.txt — 27 lines total

8. The full check inventory

Twenty deterministic checks in six families, plus one model-assisted check. The run fails if and only if a **FAIL** check fails; **WARN** and **INFO** are reported and can never change a verdict.



check	family	severity	fails when
static.syntax_valid	static	FAIL	Source does not compile. Before execution — a broken program costs no compute.
static.no_unbound_names	static	FAIL	A name is read but bound nowhere, resolved with symtable. The archived run's exact failure: hidden_dim read in forward(), assigned in no enclosing scope.
static.no_banned_calls	static	FAIL	exit() / sys.exit() would forge a clean exit code. Removes the cheapest way to fake success.
exec.exit_code_zero	exec	FAIL	The process exited non-zero. Reads the exit code, not a substring of stdout.
exec.no_uncaught_exception	exec	FAIL	An exception escaped the run. Recorded by the harness in the child process.
exec.completed_within_budget	exec	FAIL	The process group was killed on timeout.

			Kills the group, so a runaway cannot survive its own timeout.
			A traceback appears in either stream on a run that still exited 0.
<code>exec.no_swallowed_traceback</code>	<code>exec</code>	WARN	Both streams: <code>except Exception as e: print(e)</code> puts the evidence on <code>stdout</code> .
<code>env.clean_namespace</code>	<code>env</code>	FAIL	The initial globals contained anything beyond harness-provided names. Makes “the variables are real” a runtime property, not a hope.
<code>env.code_identity</code>	<code>env</code>	FAIL	The source the child hashed differs from the source the parent wrote. Makes “hashed to the run that produced it” checkable rather than assumed.
<code>env.seed_recorded</code>	<code>env</code>	WARN	No seed was declared. A run with no seed cannot be re-executed to check its own numbers.
<code>env.provenance</code>	<code>env</code>	INFO	Python version, platform, device, library versions, code SHA-256.
<code>results.contract_present</code>	<code>results</code>	FAIL	<code>results.json</code> is absent or unparseable. Nothing was recorded, so nothing is citable.
<code>results.expected_keys_present</code>	<code>results</code>	FAIL	A key the plan declared is missing.
			A recorded value is a source literal at its call site.
<code>results.values_computed</code>	<code>results</code>	FAIL	The strongest claim in Gate 1. <code>record_result("k", acc)</code> passes; <code>record_result("k", 0.816)</code> fails, as does <code>round(0.8160, 3)</code> — constant folding does not launder a typed number.
<code>results.values_finite</code>	<code>results</code>	FAIL	A value is NaN or infinite.
			A key was recorded repeatedly with a changing value.
<code>results.single_observation</code>	<code>results</code>	WARN	Every <code>record_result</code> call is retained, so a best-epoch number reported as a final-epoch one is visible.
			Exact zero, exact one, or chance level when the class count is known.
<code>results.non_degenerate</code>	<code>results</code>	WARN	Answers the limitation AutoResearchClaw reports for value registries: a registry passes real zeros.
			Numerical warnings, device failures, non-convergence, printed exceptions, ERROR-level logging.
<code>logs.no_error_signals</code>	<code>logs</code>	WARN	Tuned for precision: <code>ValueError: is flagged</code> , <code>Mean Squared Error: is not</code> .
<code>output.untruncated</code>	<code>output</code>	INFO	<code>stdout/stderr</code> byte counts; asserts no truncation was applied. Retires the 1,000-character ceiling.
<code>report.fixes_grounded</code>	<code>report</code>	INFO	Generated fixes cited something the run does not contain. Records that the model's text was discarded and why.
<code>logs.model_error_signals</code>	<code>logs (model)</code>	WARN	Lines the pattern set could not reach, read by the LLM layer. WARN by construction — <code>model_warning()</code> takes no severity parameter,

so no model output can reach the verdict.

Why the severity split is load-bearing. The LLM layer is a required part of Gate 1 — the feedback report is the loop's return path to the ML engineer, and no template writes “bind hidden_dim before use, or pass it into GCN.__init__”. It coexists with a model-free verdict because of where it sits and what severity it can emit: model findings are WARN by construction, and report generation runs after `decide()` has already fixed the verdict. A test parses the module's AST to assert `Severity.FAIL` never appears in an expression there.

9. Metrics the gate records per attempt

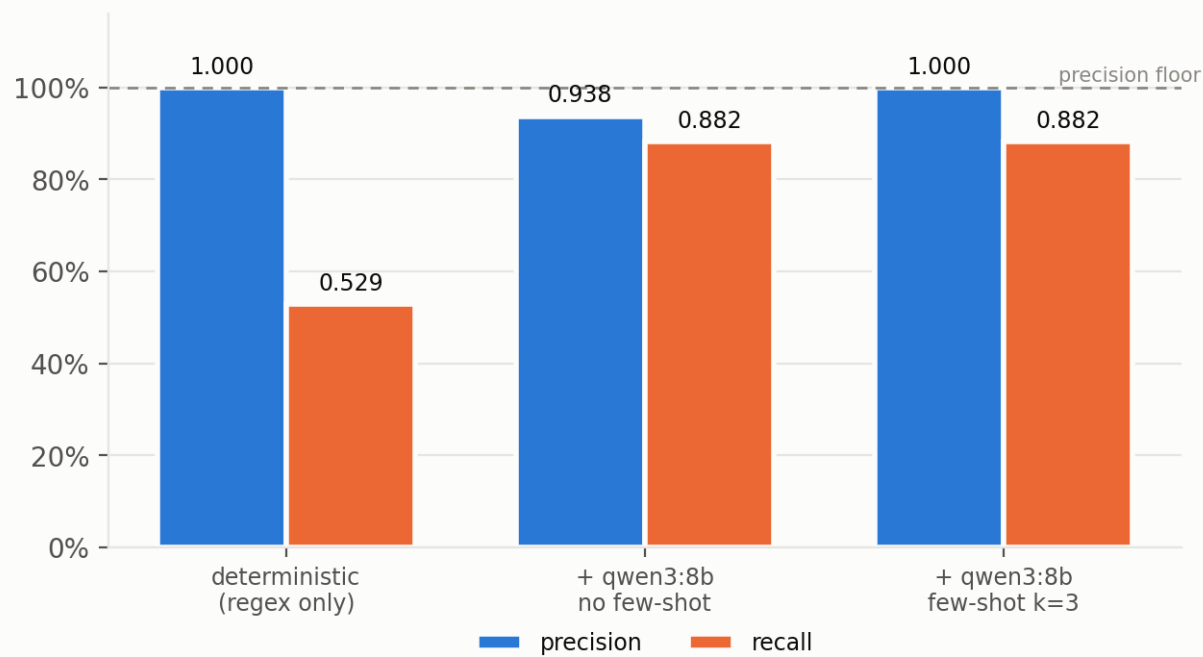
metric	definition
trace_id	<code>sha256(run_id, key, lineno)</code> — binds one value to one execution. Two runs of identical source give different trace ids, which is what makes a backfilled number detectable.
chain_integrity	fraction of values whose provenance chain (task → command → log → value) is complete, with the missing link named per key.
citable	false on a rejected run's registry, so a consumer that ignores the verdict still cannot cite it.
arg_kind	<code>computed</code> or <code>literal</code> , from static analysis of the <code>record_result</code> call site.
call_count	times a key was recorded, with the value <code>span</code> — a best-epoch number reported as final is visible.
code_sha256	hashed by the child that ran it and compared to what the parent wrote.
<code>model.calls</code> / degraded	what the LLM layer spent, and whether any call failed, so a thinner report is never mistaken for a complete one.

10. The log scanner, measured

68 labelled lines, 34 of them error signals, 16 of those outside anything a regex was going to reach. Sixteen lines come verbatim from the archived run.

Log-scanner accuracy on the 68-line labelled corpus

Precision is the floor: a false positive puts a non-issue into the paper



scanner	precision	95% CI	recall	95% CI
deterministic (regex only)	1.000	0.824–1.000	0.529	0.367–0.685
+ qwen3:8b, no few-shot	0.938	0.798–0.983	0.882	0.734–0.953
+ qwen3:8b, few-shot k=3	1.000	0.886–1.000	0.882	0.734–0.953

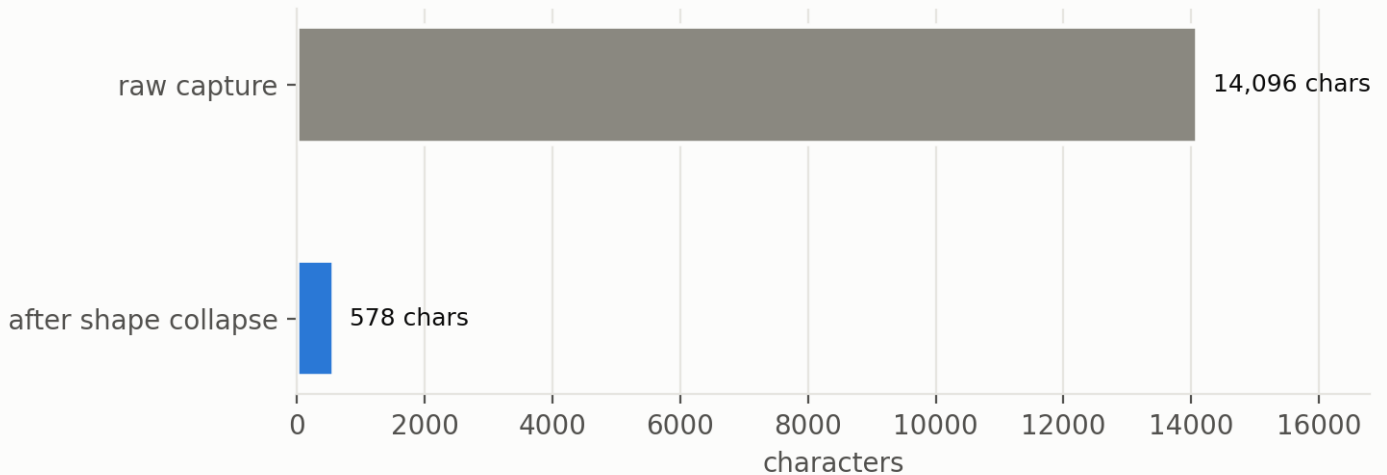
The model tier raised recall from 0.529 to 0.882 at no cost in precision (1.000 → 1.000), taking F1 from 0.692 to 0.938. It runs on a local 8B model at zero marginal cost, so this is applied to every attempt rather than sampled.

Retrieval is what bought the precision back. Without few-shot examples the same model scored 0.938 precision and flagged arch-009, arch-014 — framework noise, one of them the cuDNN registration notice the corpus was built around. Balanced retrieval (k signals and k negatives, never whichever scores highest) removed both while recall held at 0.882. The nearest *negative* was the useful neighbour, which is the whole reason the exemplar bank is half noise.

Still missed at k=3: arch-004, dev-004, dev-005, gap-007 — 4 of 34 signals.

Prompt sent to the log scanner — 95.9% smaller

203 non-blank lines collapse to 4 distinct shapes; lossless in distinct content

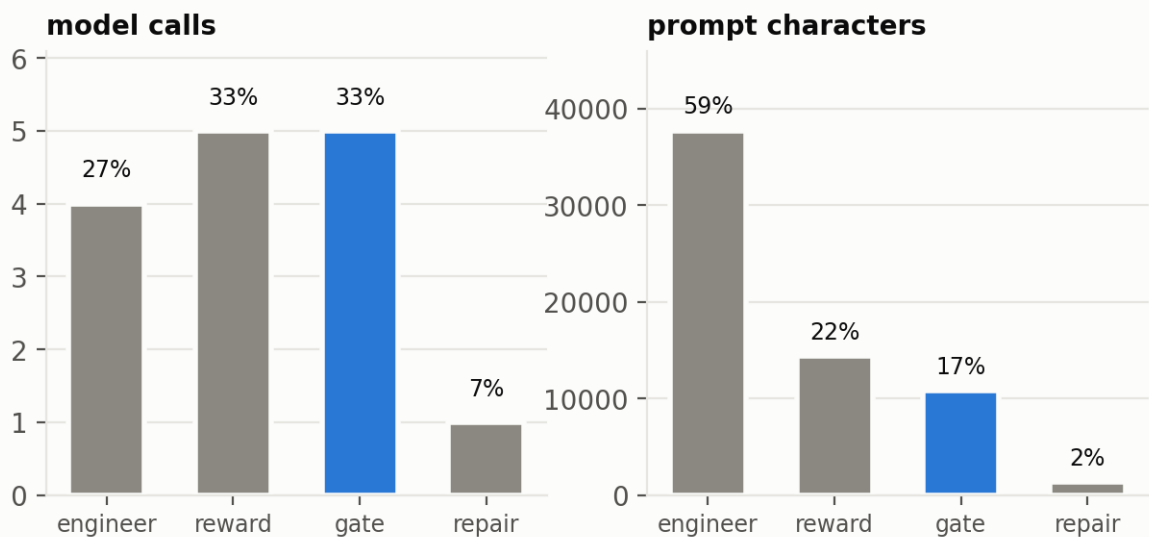


Before the model reads anything, the log is collapsed to its distinct shapes: **203 non-blank lines to four**, a 95.9% smaller prompt. The collapse is lossless in distinct content — every shape survives with its first real line number, so nothing a scanner could have flagged disappears.

11. What the layer costs in a phase

Gate 1 is a third of the calls and a sixth of the tokens

Measured over five executions of one solver phase; blue is the gate



Measured over five executions of one solver phase: Gate 1's own model calls are **a third of the calls but a sixth of the tokens**. That is why the gate can run on qwen3:8b on an 8 GB laptop GPU — 5.5 GB resident, ~16s a call, zero marginal cost — while the engineer stays on a stronger model. Putting the *engineer* on a weak model is the tempting economy and the wrong one: it fails the gate more often, and every rejection costs a fixed call, a shadow-reward call and another turn.

12. Defects the runs found

Nine, fixed. Four appeared only against a real model, and two of those only against the weaker one.

defect	found by	consequence
Unparseable reply executed an empty experiment	scripted e2e	Gate reported “you never called record_result()” when the command had not parsed
Evidence bundle showed warning counts without evidence	scripted e2e	The writer was told to disclose warnings it could not see
Fallback note reported an outage that had not happened	scripted e2e	Three different situations under one message
Prose in backticks parsed as code	gemini-3.5-flash	A good fix discarded over ['so', 'that', 'it', 'when', 'executing']
Prompt leaked the gate's artifact paths	gemini-3.5-flash	Told the engineer to edit /home/.../attempt_03/experiment.py
Scan prompt offered two numbers per row	qwen3:8b	The weaker model reported the file line, not the row index; a correct finding was discarded
Output cap unreachable for most backends	qwen3:8b	One call generated 21,858 tokens for an answer whose correct form is []
Rate-limit retry killed the phase in 25s	gemini-3.5-flash	Flat 5s × 5 against a per-minute meter
Five config keys reached nothing	integration audit	A config declaring a lit-review backend did not get one

13. This run, scored against MLR-Bench's taxonomy

Our run, their categories. This is **not** a run of MLR-Bench — that needs their harness and their task set — but the first of their four classes is measurable on any run, and here it is measured rather than asserted.

MLR-Bench class	with Gate 1	without Gate 1	whose job
Faked experimental results Data, metrics or outcomes fabricated or never performed	0 of 6 unbacked; 6 reproduced on re-execution	every number it reports is unbacked — it recorded none through any contract, so the writer receives prose to copy from	measured — this is the class Gate 1 addresses
Hallucinated methodology Techniques claimed but not implemented	not measured here	not measured here	Gate 2 — needs the plan compared to the code
Incorrect citations References wrong or nonexistent	not measured here	not measured here	Gate 3 — the manuscript is out of Gate 1's scope

Mathematical errors

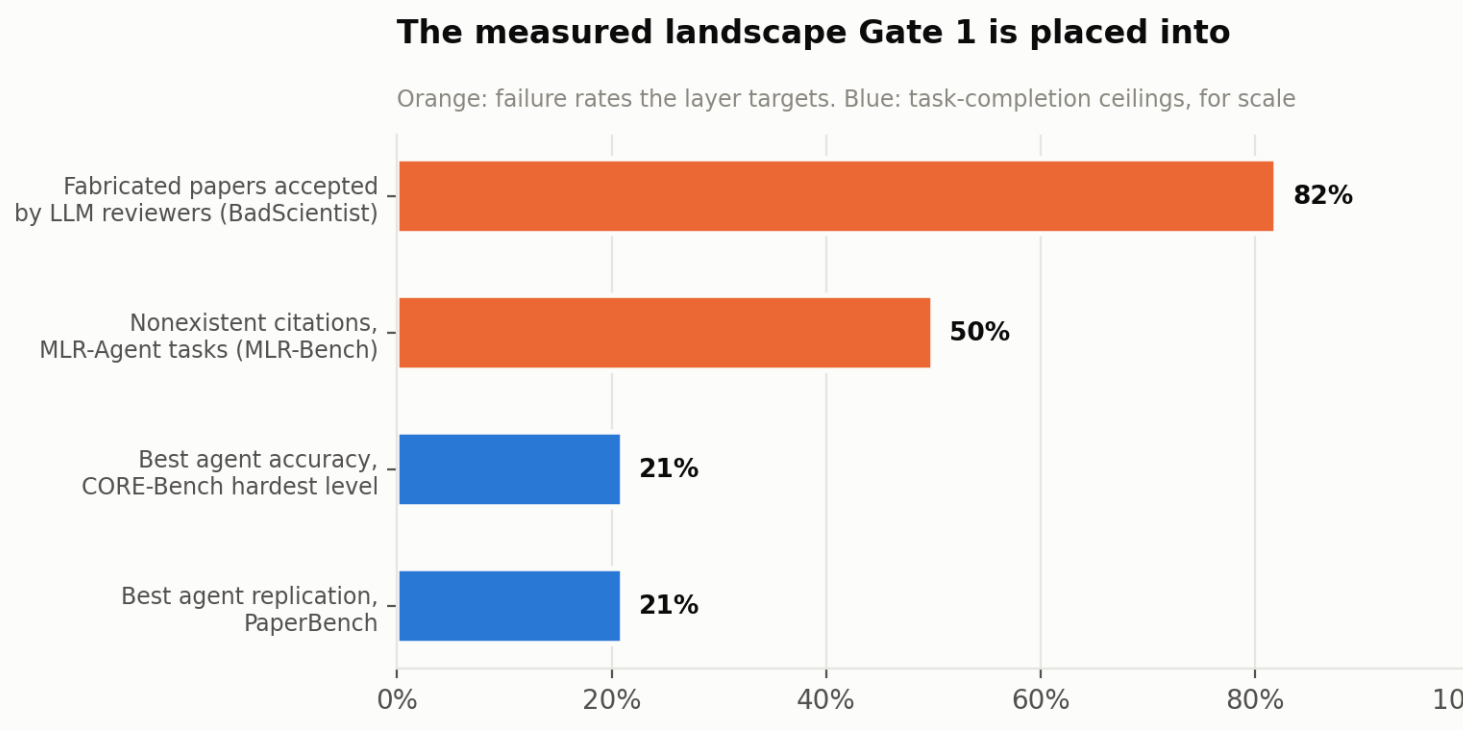
Wrong equations or derivations

not measured here

not measured here

outside all three gates as specified

14. Where this sits against the published benchmarks



source	what it measures	reported
MLR-Bench arXiv 2505.19955	Four fact-based hallucination types in AI-generated papers.	Faked results and hallucinated methodology each in more than half of 10 tasks; almost all AI Scientist V2 papers contain both . Nonexistent citations in 50% of MLR-Agent tasks.
BadScientist arXiv 2510.18003	Whether fabricated papers pass multi-model LLM review.	Accepted at up to 82.0% ; detection “barely exceeding random chance”.
CORE-Bench arXiv 2409.11363	Computational reproducibility by agents.	Best agent 21% on the hardest level; correctness judged against a 95% prediction interval over three manual reproductions.
PaperBench arXiv 2504.01848	Replicating AI research end to end.	Best agent 21.0% ; ML PhDs 41.4% best-of-3 after 48 hours.

BadScientist is the reason Gate 1's verdict consults no model: if LLM reviewers detect fabrication at barely above chance, a validity layer built on model judgement inherits that ceiling.

Naming. This project's design documents paraphrase MLR-Bench's taxonomy. Its published names are *faked*

experimental results, hallucinated methodology, incorrect citations, mathematical errors. “Silent failure scored as success” is a cause of the first in their scheme, not a fifth class, and any “eliminates N of four” claim should say so.

15. Where every number here came from

Each quantity below is either a measurement with a file behind it, a figure quoted from a paper, or narrative. The third category is listed rather than hidden, because a report about fabricated results should not contain any of its own.

quantity	source	kind
Per-run arm results, reproduction rates, false successes	5 × ablation.json on disk	measured
Token and cost figures	usage block of each ablation.json	measured
Per-check outcomes per turn	gate1_report.json per attempt	measured
Log excerpts and paths	stdout.txt / stderr.txt per attempt	measured
Log-scanner precision and recall	rig/corpus.py over tests/fixtures/log_corpus.jsonl	measured
Prompt compression 203 lines to 4 shapes	gates/log_digest.py on a recorded capture	measured
Check inventory counts	counted from gates/gate1.py	measured
MLR-Bench, BadScientist, CORE-Bench, PaperBench figures	quoted from the papers, arXiv ids given	cited
Archived-run figures: 81.60%, 13.61x, reward 1.0	results/gemini_3_5_flash_run_1 log in the host repo	measured
Defect list and the story of how each was found	this session's commit history	narrative — not a measurement
Report completeness, traceability and reproduction, both arms	rig/report_accuracy.py over every attempt on disk; ungated arm re-executed to check	measured

16. What Gate 1 does not claim

- It does not check whether a result is *plausible* — that is Gate 2.
- It does not read the manuscript — that is Gate 3.
- The static tier is conservative on purpose; the runtime tier catches what it declines to claim, one execution later.
- The scanner's recall is bounded by the lines it was shown, and that count is recorded so the claim cannot exceed the evidence.

- A mean computed inside the experiment over a silently shortened list arrives as one value the gate cannot see behind.
- The ablation is **n = 1 at temperature 0**. It shows what happened on one task with one model; it is not a rate.
- n is still 1 for the archived-run comparison — the re-run has not been done.

17. Reproducing this

```
./tools_local_model.sh start          # local qwen3:8b, no API key
python tools_ablation.py --model qwen3-8b-local --turns 4
python -m rig.corpus                  # deterministic scanner baseline
python -m rig.gate1_loop              # the five loop scenarios
python reports/make_charts.py && python reports/make_report.py
python reports/make_deck.py
pytest                               # 249 gate tests
```

Sources: MLR-Bench (2505.19955), BadScientist (2510.18003), CORE-Bench (2409.11363), PaperBench (2504.01848), RE-Bench (2411.15114), AutoResearchClaw (2605.20025), ScientistOne (2605.26340), SAGE (2606.31478).